第二部分：拆包

Slide 14: 什么是拆包?

- 定义：将序列中的元素分配给多个变量。
- 示例：

```python
a, b, c = [1, 2, 3]
print(a, b, c)
```

Slide 15: 多个元素对应一个变量

- 使用 `*` 接收多余的元素。
- 示例：

```python
*a, b, c = [1, 2, 3, 4, 5, 6]
print(a, b, c)
```

Slide 16: 打散

- 使用 `*` 打散序列。
- 示例：

```python
print(*[1, 2, 3])
```

第三部分：函数返回值

Slide 17: 什么是返回值?

- 定义：函数处理的结果，使用 `return` 返回。
- 示例：

```python
def factory(a):
    return a + 1
result = factory(1)
print(result)
```

Slide 18: return 的特性

1. 结束函数运行。
2. 返回值无类型限制。
3. 返回值无个数限制（多个值返回元组）。
4. 默认返回 `None`。

Slide 19: 多个返回值

- 示例：

```python
def factory(a):
    return [1, 2, 3], True, 'aaaa'
result = factory(1)
print(result)
```

Slide 20: return 结束循环

- 示例：

```python
def factory(a):
    while True:
        if a == 3:
            return
        a += 1
        print(a)
factory(1)
```

Slide 21: 总结

- 函数是Python中的核心概念，用于封装代码逻辑。
- 参数和返回值使函数更加灵活和强大。
- 拆包和打散是Python中常用的操作，简化了数据处理。

第四部分：函数调用的三种方式

Slide 1: 函数调用的三种方式概述

- 标题：函数调用的三种方式
- 内容：
  - 1. 语句形式
  - 2. 表达式形式
  - 3. 作为参数传递给其他函数

Slide 2: 语句形式

- 标题：语句形式
- 内容：
  - 代码示例：

    python 复制
    ```python
    def foo():
        print('from foo')
    foo()
    ```
  - 解释：
    - 直接调用函数，执行函数体中的代码。

Slide 3: 表达式形式

- 标题：表达式形式
- 内容：
  - 代码示例：

    python 复制
    ```python
    def foo(x, y):
        res = x + y
        return res
    res1 = foo(1, 2) * 100
    print(res1)
    ```
  - 解释：
    - 函数调用作为表达式的一部分，返回值可以参与运算。

Slide 4: 作为参数传递给其他函数

- 标题：作为参数传递给其他函数
- 内容：
  - 代码示例：

    python 复制

    ```python
    def max2(x, y):
        if x > y:
            return x
        else:
            return y
    print(max2(max2(1, 2), max2(4, 5)))
    ```

  - 解释：
    - 函数调用可以作为参数传递给另一个函数。

第五部分：函数局部变量与全局变量的修改

Slide 5: 局部变量与全局变量概述

- 标题：局部变量与全局变量
- 内容：
  - 局部变量：函数内部定义的变量，只能在函数内部访问。
  - 全局变量：函数外部定义的变量，可以在整个模块中访问。

Slide 6: 外部不能访问函数内部的变量

- 标题：外部不能访问函数内部的变量
- 内容：
  - 代码示例：

    python 复制

    ```python
    def fun1():
        x = 1
    print(x)  # 报错
    ```

  - 解释：
    - 函数内部的变量（局部变量）不能在函数外部访问。

Slide 7: 函数内部可以访问外部变量

- 标题：函数内部可以访问外部变量
- 内容：
  - 代码示例：

    python 复制

    ```python
    x = 123
    def fun2():
        print(x)
    fun2()
    ```

  - 解释：
    - 函数内部可以访问函数外部的全局变量。

Slide 8: 函数内部不能修改外部变量

- 标题：函数内部不能修改外部变量
- 内容：
  - 代码示例：

    python 复制

    ```python
    x = 123
    def fun2():
        x = x + 1  # 报错
    fun2()
    ```

  - 解释：
    - 函数内部默认不能修改函数外部的全局变量。

Slide 9: 使用 global 修改全局变量

- 标题：使用 global 修改全局变量
- 内容：
  - 代码示例：

    python 复制

    ```python
    x = 123
    def fun2():
        global x
        x = x + 1
        print(x)
    fun2()
    ```

  - 解释：
    - 使用 `global` 关键字可以在函数内部修改全局变量。

Slide 10: 使用 nonlocal 修改嵌套函数变量

- 标题：使用 nonlocal 修改嵌套函数变量
- 内容：
  - 代码示例：

    python 复制

    ```python
    def func2():
        b = 100
        def func3():
            nonlocal b
            b += 1
            print(b)
        func3()
    func2()
    ```

  - 解释：
    - 使用 `nonlocal` 关键字可以在嵌套函数中修改外层函数的变量。

Slide 11: 名称空间与作用域

- 标题：名称空间与作用域
- 内容：
  - 名称空间：存放数据的空间，分为全局、局部和内置名称空间。
  - 作用域：访问名称空间的权限，分为全局作用域、局部作用域和内置作用域。
  - 类比：
    - 名称空间：储物柜（存放数据）。
    - 作用域：钥匙（访问权限）。

Slide 12: 总结

- 标题：总结
- 内容：
  - 函数调用的三种方式：语句形式、表达式形式、作为参数传递。
  - 局部变量与全局变量的访问与修改：`global` 和 `nonlocal` 的使用。
  - 名称空间与作用域的关系：名称空间存放数据，作用域决定访问权限。